

AN2DL - First Challenge Report

theoverfitters

Raul Agolli, Kevin Pio Abate, Andrea Ascenzo Mastroberti

raulagolli, keevin16, drebishh

10783671, 10812892, 10736595

November 17, 2025

1 Introduction

The goal of this challenge is to determine each subject's true pain status from multivariate time series data. The dataset, *The Pirate Pain Dataset*, captures temporal patterns of body joints and pain perception from both ordinary individuals and pirates. The target classes are **no_pain**, **low_pain**, and **high_pain**.

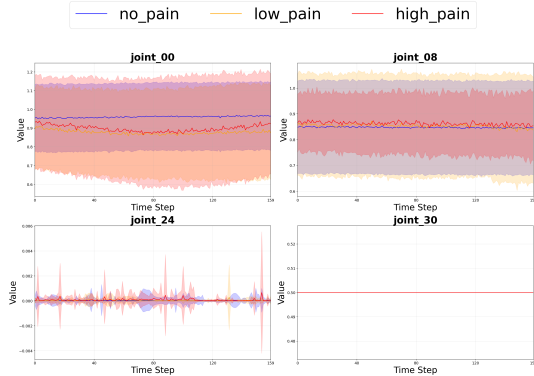


Figure 1: Mean and standard deviation by pain class of some joint features

2 Problem Analysis

The dataset presents several challenges inherent to time-series classification, our initial focus was on an-

alyzing available techniques to address them effectively.

2.1 Challenges

The main challenges of this study are the **limited size** of the dataset and its **imbalance**.

The dataset contains only 660 pirates' samples with 160 timesteps and 40 features, creating a high-dimensional setting where larger models are prone to overfitting. To address this, we performed several preprocessing steps.

- **Pain survey consolidation:** The four pain-assessment series were merged into a single integer feature by taking the median and flooring the result. The original columns were removed.
- **Morphological feature merging:** The categorical features `n_legs`, `n_hands`, and `n_eyes` were compressed into a binary flag (`pirate`), set to 0 for normal subjects and 1 for malformed ones.
- **Constant feature removal:** The feature `joint_30`, which had a constant value of 0.5 across all samples, was dropped.
- **2D Sinusoidal Positional Encoding:** To facilitate the model's recognition of cyclical dependencies and avoid linear time index

boundary effects, a 2D sinusoidal positional encoding (PE) was implemented. Utilizing a **PERIOD** = 80, the original **time** feature was replaced by two continuous variables:

$$\begin{aligned}\text{time_sin} &= \sin\left(\frac{2\pi \cdot \text{time}}{80}\right) \\ \text{time_cos} &= \cos\left(\frac{2\pi \cdot \text{time}}{80}\right)\end{aligned}$$

This embedding maps time onto a unit circle, explicitly modeling temporal proximity across the period boundary.

- **Scaling:** Continuous features were scaled using *MinMax Scaling*. To handle varying sequence lengths and augment the data, we employed a sliding window approach with tunable window sizes and strides.

3 Method

We began our experiments with a simple RNN model to establish a baseline for our approach. Our initial focus was on preprocessing and augmenting the dataset to improve model performance.

3.1 Data Augmentation

We experimented with various augmentation techniques, including Generative Adversarial Network (GAN), Gaussian noise and Time Warping. However, these methods did not yield any significant improvement in performance. Consequently, we decided to discontinue their use and to work on the original data. Our focus switched on other aspects of the pipeline.

3.2 Network Architecture Exploration

Given the limited success with the initial network, we decided to explore stronger models without altering the dataset further. We experimented the RNN with windowing, a **Dual-Model** strategy where we trained two specialized binary classifiers, M1 for classifying no pain vs. pain (combining low and high), and M2 Classifies low pain vs high pain (trained only on pain samples). However, these were too large and complex, often resulting in poor validation performance, likely due to overfitting.

Afterward, we attempted to implement a custom network inspired by the CSHN[1].

3.3 Implementation and Customization

Our best-performing model consists of a three-layer 1D CNN, combining **convolutional** and **normalization** layers, followed by a **Global Average Pooling** (GAP) layer to reduce the dimensionality of the feature maps before classification. This structure helps maintain a compact set of parameters and contributes to improved computational efficiency. The CNN output is then passed to a **Recurrent Neural Network** (LSTM) to capture temporal dependencies within the time series.

After that the **Attention Layer**: was employed to significantly enhance the network’s ability to focus on informative features by adaptively recalibrating the importance of inputs across the temporal sequence.

3.4 Class Imbalance Resolution

A significant improvement was made when, analyzing the class distribution, we discovered that the low_pain and high_pain classes were too small for our classification scope. As a result, we adjusted the class weights to be inversely proportional to their frequencies. This adjustment led to a drastic improvement of the performance on both the validation and test datasets.

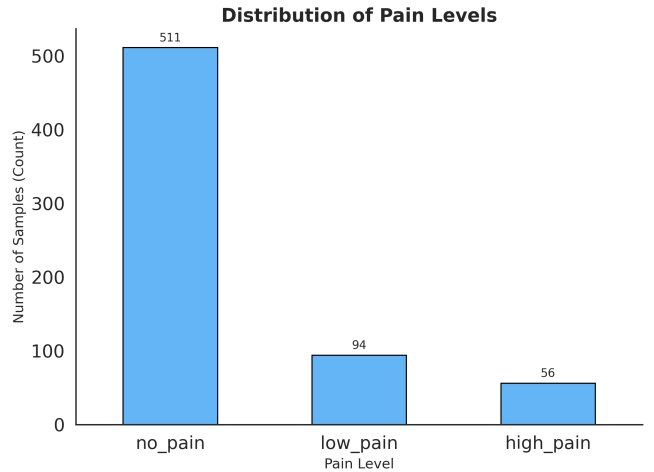


Figure 2: Distribution of the classes on the unbalanced dataset

3.5 Loss Function

We explored several loss functions and tuned their associated parameters to assess their effectiveness. Among the tested options, **weighted Cross-Entropy** provided the most reliable performance. In contrast, Focal Loss did not lead to measurable improvements, and the application of label smoothing consistently degraded results.

We attempted to solve frequent high-pain/no-pain misclassifications by applying a penalty to those mispredictions in the cross entropy as well, with mild improvements.

4 Experiments

We used **PyTorch** for model implementation and **Optuna** for hyperparameter tuning.

4.1 Hyperparameter Tuning

We dedicated the majority of our training experiments to hyperparameter optimization, with Optuna, carefully assessing the impact of each adjustment in order to guide the model toward the desired performance.

We assessed the model using 4-Fold Cross-Validation with fold-level multi-GPU parallelization and pruning to speed computation.

The best parameters (Trial 71) were:

Window Size: 20 time steps

Stride: 5 time steps (high overlap for augmentation)

CNN Filters: $8 \rightarrow 16 \rightarrow 32$

RNN: LSTM, 1 Layer, Hidden Size 8

Dropout: CNN 0.26, RNN 0.59

Optimizer: AdamW with $lr = 3.5e^{-4}$ and weight decay $2.2e^{-5}$

4.2 Training Protocol

To ensure robustness, the best configuration was re-trained using **Stratified Group 10-Fold Cross-Validation**. This prevents data leakage by ensuring all windows from the same subject remain in the same fold. We used Standard Cross Entropy Loss (weighted) and Early Stopping.

5 Results

The model demonstrated high stability across the 10 folds. The aggregated validation results are presented below.

Table 1: 10-Fold CV Aggregated Metrics (Best Epoch)

Metric	Value
Accuracy	0.9218
F1-Score (Macro)	0.8316
F1-Score (Weighted)	0.9207

6 Discussion

The success of the single hybrid model over the dual-model ensemble suggests that the feature space is most likely better learned jointly. The sliding window with a small stride (5) was crucial, significantly increasing the effective training set size and allowing the small LSTM (hidden size 8) to generalize well without overfitting. The static branch provided a necessary bias correction for the "pirate" subjects, who likely exhibit different movement baselines.

7 Conclusions

We successfully developed a robust pipeline for pain classification using a Hybrid CNN-LSTM architecture with a static feature branch. The key contributions were the explicit modeling of morphological data and the aggressive windowing strategy tuned via Optuna. The final model achieves a weighted F1-score of **0.92** in cross-validation.

References

- [1] Wang, X., Zhang, Y., Bai, N., Yu, Q., & Wang, Q. (2024). Class-imbalanced time series anomaly detection method based on cost-sensitive hybrid network. *Expert Systems With Applications*, 238, 122192.